Signals and Systems

Computer Assignment 2

Report

Parsa Taherkhani

810102579

Part 1 Report: English License Plate Character Extraction

1. Introduction

This part of the project implements a simple license-plate character extraction and recognition pipeline in MATLAB without relying on high-level built-in functions for most core operations. The pipeline reads a color image of a vehicle plate, resizes it, converts to grayscale, thresholds to a binary image, removes small/large connected components, segments the image into connected components, and classifies each extracted character by correlating with a provided dataset of character templates.

The goal is to demonstrate basic image-processing building blocks (color-to-gray conversion, binary thresholding, connected-component analysis, segmentation) and a simple template-matching classifier.

2. Data and Tools

- MATLAB (scripts and functions written in native MATLAB).
- Training set: data/English_Map_Set — image files for English alphanumeric characters used as templates.
- Output: english_plate_result.txt contains the final recognized characters (one line).
- Scripts / functions used in Part 1:
  - `p1.m` (main script)
  - `mygrayfun.m` (grayscale conversion)
  - `mybinaryfun.m` (binary thresholding)
  - `myremovecom.m` (remove components smaller than n pixels)
  - `mysegmentation.m` (connected component segmentation / labeling)

3. Algorithm / Implementation

1. File selection & resizing: The user selects an image and it's resized to [300 500].
2. Grayscale conversion (mygrayfun): 0.299R + 0.578G + 0.114B.
3. Binary thresholding (mybinaryfun): Converts image ≤ threshold (100) to 1, else 0.
4. Connected-component removal (myremovecom): Keeps components ≥ n pixels; subtracting myremovecom(image,300) - myremovecom(image,2300) keeps medium-sized ones.
5. Segmentation (mysegmentation): Manual flood-fill 8-connected labeling.
6. Character extraction and sorting: Each labeled region is cropped and sorted by x-coordinate.
7. Classification: Each cropped character is compared to templates with corr2 ≥ 0.45; the best match is chosen.

4. Function Details

`mygrayfun`: Converts color to grayscale.
`mybinaryfun`: Thresholds grayscale image.
`myremovecom`: Removes small components.
`mysegmentation`: Implements custom 8-connected labeling using a stack-based flood-fill.

6. Results

The script displays original, grayscale, binary, and cleaned images. It also shows each segmented character and prints the recognized sequence in the MATLAB console and english_plate_result.txt.

7. Discussion and Limitations

Strengths:
- Full segmentation implemented manually without `bwlabel`.
- Simple, interpretable correlation-based recognition.

Limitations:
- Fixed thresholding; not adaptive.
- Sensitive to lighting and font.
- Area thresholds are image-dependent.
- No morphological cleanup.
- Correlation limited to same-size templates.

Improvements:
- Adaptive thresholding.
- Morphological preprocessing.
- Feature-based or ML classification (HOG, SVM, etc.).
- Skew correction before segmentation.

8. Conclusion

The implemented pipeline demonstrates core image-processing and recognition steps for English license plates. The modular design simplifies experimentation. For robust performance across varied inputs, adaptive binarization and more advanced classification are recommended.

1-1

- Script

```
[file,path] = uigetfile({'*.jpg;*.png'},'Select your image');
image = imread([path file]);
```

1-2

Script

```
image = imresize(image, [300 500]);

figure;
subplot(2,2,1)
imshow(image)
title('Base')
```

Result



1-3

Implementation of `mygrayfun`

```
function gray_image = mygrayfun(image)

    gray_image = 0.299 * image(:,:,1) + 0.578 * image(:,:,2) + 0.114 * image(:,:,3);

end
```

Script

```
subplot(2,2,2)
image = mygrayfun(image);
imshow(image)
title('Gray')
```

Result



1-4

Implementation of `mybinaryfun`

```
function binary_image = mybinaryfun(image, treshold)

    binary_image = image <= treshold ;

end
```

Script

```
subplot(2,2,3)
image = mybinaryfun(image, 100);
imshow(image)
title('Binary')
```

Result



1-5

implementation of `myremovecom`

- **Script**

```
function cleared_image = myremovecom(image, n)

    [L, num] = mysegmentation(image);
    cleared_image = false(size(image));

    for label = 1:num
        component = (L == label);
        if sum(component(:)) >= n
            cleared_image = cleared_image | component;
        end
    end

end
```

Script

```
subplot(2,2,4)
image = myremovecom(image,300)-myremovecom(image,2300);
imshow(image)
title('Cleaned')
```

Result



1-6

implementation of `mysegmentation`

- **Script**

```
function [L, n] = mysegmentation(image)
    [rows, cols] = size(image);
    L = zeros(rows, cols);
    current_label = 0;

    directions = [-1,  0; 1, 0; 0, -1; 0, 1; -1, -1; -1, 1; 1, -1; 1, 1];

    for r = 1:rows
        for c = 1:cols
            if image(r, c) == 1 && L(r, c) == 0
                current_label = current_label + 1;
                stack = [r, c];

                while ~isempty(stack)
                    pixel = stack(end, :);
                    stack(end, :) = [];
                    r_pixel = pixel(1);
                    c_pixel = pixel(2);

                    if r_pixel < 1 || r_pixel > rows || c_pixel < 1 || c_pixel > cols || L(r_pixel, c_pixel) ~= 0 || image(r_pixel, c_pixel) == 0
                        continue;
                    end

                    L(r_pixel, c_pixel) = current_label;

                    for i = 1:size(directions, 1)
                        new_r = r_pixel + directions(i, 1);
                        new_c = c_pixel + directions(i, 2);
                        stack = [stack; new_r, new_c];
                    end
                end
            end
        end
    end

    n = current_label;
end
```

Result



1-7

Script

```
[L,maxSegment] = mysegmentation(image);
maxSegment = maxSegment+1;
hold on

label_positions = zeros(2,maxSegment-1);
for i = 1:maxSegment-1
    [r,c] = find(L == i);
    mc = min(c,[],'all');
    mr = min(r,[],'all');
    Mc = max(c,[],'all');
    Mr = max(r,[],'all');
    label_positions(1,i) = i;
    label_positions(2,i) = mc;
    rectangle('Position',[mc,mr,Mc-mc,Mr-mr],'EdgeColor','b','LineWidth',2)
end

[temp,indexs] = sort(label_positions(2,:));
label_positions(1,:) = label_positions(1,indexs);
label_positions(2,:) = label_positions(2,indexs);

files=dir('English_Set');
len=length(files)-2;
TRAIN=cell(2,len);
for i=1:len
   TRAIN{1,i}=imread(['English_Set','/',files(i+2).name]);
   TRAIN{2,i}=files(i+2).name(1);
end

figure
result = [];
```

```
for i = 1:maxSegment-1
    [r,c] = find(L == label_positions(1,i));
    character = image(min(r):max(r), min(c):max(c));
    subplot(1,maxSegment-1,i)
    imshow(character)
    decision = cell(2,1);
    decision{1,1} = 0;
    decision{2,1} = '';

    for j = 1:length(TRAIN)
        temp = corr2(imresize(TRAIN{1,j},size(character)), character);
        if temp >= 0.45 && temp >= decision{1,1}
            decision{1,1} = temp;
            decision{2,1} = TRAIN{2,j};
        end
    end

    result = [result decision{2,1}];
end

disp(result)
file = fopen('plate_result.txt', 'wt');
fprintf(file,'%s\n',result);
fclose(file);
```

1-8

Script

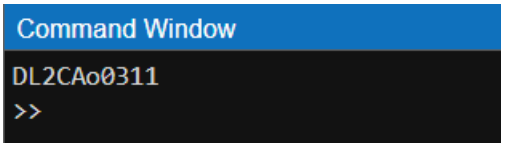
```
disp(result)
file = fopen('plate_result.txt', 'wt');
fprintf(file,'%s\n',result);
fclose(file);
```

Result

Content of the `english_plate_result.txt`



Command line



Part 2

1. Introduction

In Part 2, the same license-plate character extraction and recognition pipeline is implemented for Persian license plates. This time, MATLAB built-in functions for image processing (`rgb2gray`, `imbinarize`, `bwlabel`, `bwareaopen`) are used to simplify the process compared to Part 1, where all operations were implemented manually. The aim is to compare both approaches and evaluate the accuracy and performance differences.

2. Data and Tools

- MATLAB built-in image processing functions.
- Training set: data/Persian_Map_Set — includes Persian character templates.
- Output: persian_plate_result.txt.
- Script used in Part 2: p2.m (main script).

3. Algorithm / Implementation

1. File selection & resizing: The selected image is resized to [300 500].
2. Grayscale conversion: Performed with rgb2gray(image).

3. Binary conversion: Using imbinarize, followed by logical negation (~image) to ensure black characters on white background.
4. Component filtering: Uses bwareaopen(image,400) - bwareaopen(image,5000) to retain objects between 400 and 5000 pixels in area.
5. Segmentation: Connected components labeled using bwlabel.
6. Sorting: Components are sorted left to right based on their x-coordinates.
7. Character extraction: Each labeled region is cropped using bounding boxes.
8. Recognition: Each extracted character is compared to training templates using corr2 ≥ 0.45; the best match is appended to result.
9. Output: Recognized characters are displayed in console and written line-by-line to persian_plate_result.txt.

4. Built-in Functions

- rgb2gray: Converts RGB image to grayscale automatically.
- imbinarize: Adaptive global thresholding for binarization.
- bwareaopen: Removes small connected components below a given pixel threshold.
- bwlabel: Labels connected components (4-connected by default).

5. Parameters

- Resize: [300 500].
- bwareaopen thresholds: 400 (remove small objects), 5000 (remove large objects).

- corr2 threshold: 0.45 for character matching.
- Output file: persian_plate_result.txt.

6. Results

The code successfully segments and recognizes Persian characters from license plates. It displays the original, grayscale, binary, and cleaned versions, as well as cropped characters in a separate figure. The recognized characters are printed in the console and saved to persian_plate_result.txt.

Persian dataset



```
clear;
clc;
close all;

[file,path] = uigetfile({'*.jpg;*.png'},'Select your image');
image = imread([path file]);
image = imresize(image, [300 500]);

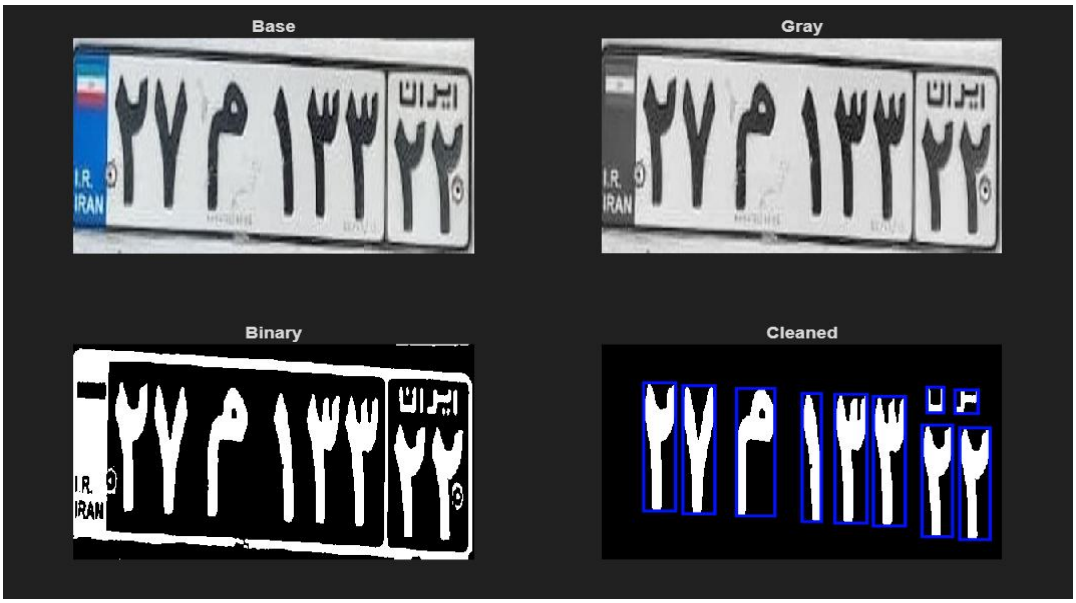
figure;
subplot(2,2,1)
imshow(image)
title('Base')

subplot(2,2,2)
image = rgb2gray(image);
imshow(image)
title('Gray')

subplot(2,2,3)
image = imbinarize(image);
image = ~image;
imshow(image)
title('Binary')

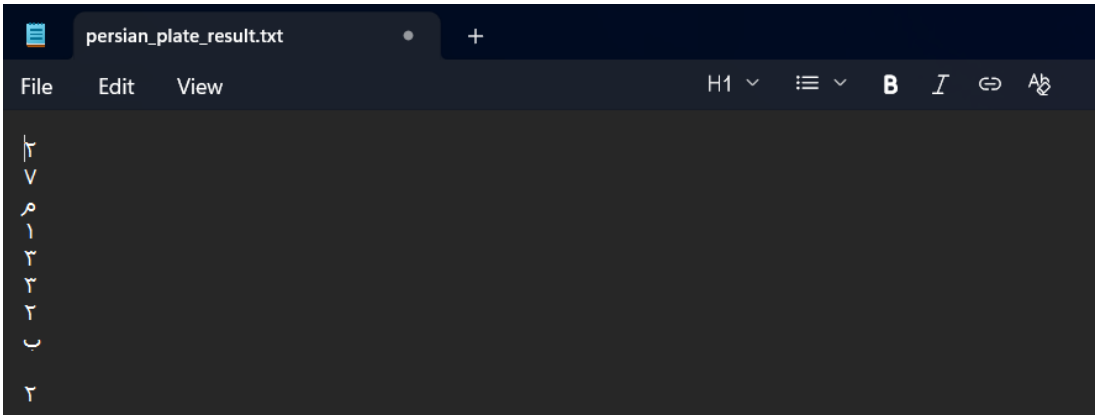
subplot(2,2,4)
image = bwareaopen(image,400) - bwareaopen(image,5000);
imshow(image)
title('Cleaned')
```

Result

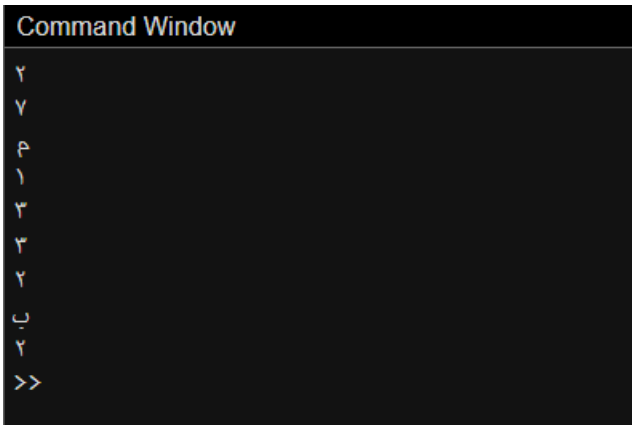




Content of the `persian_plate_result.txt`



Command line



Part 3







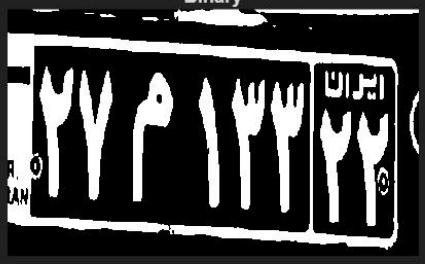
Base



Gray

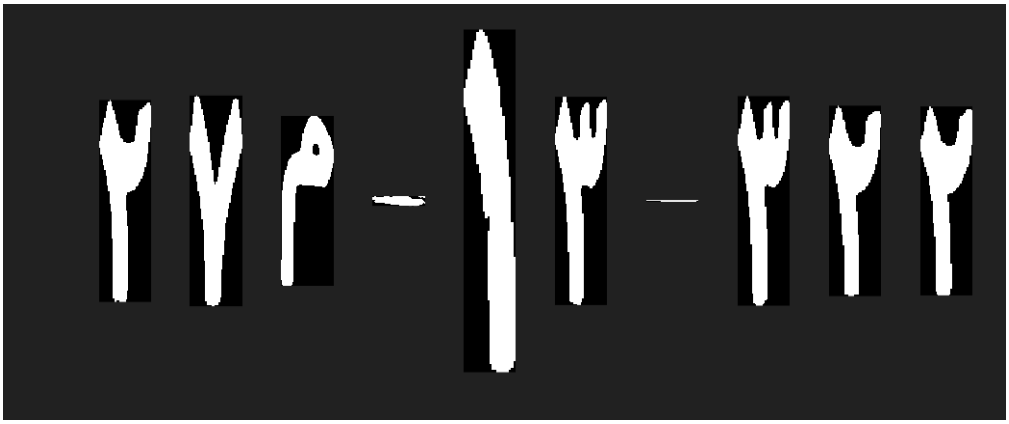


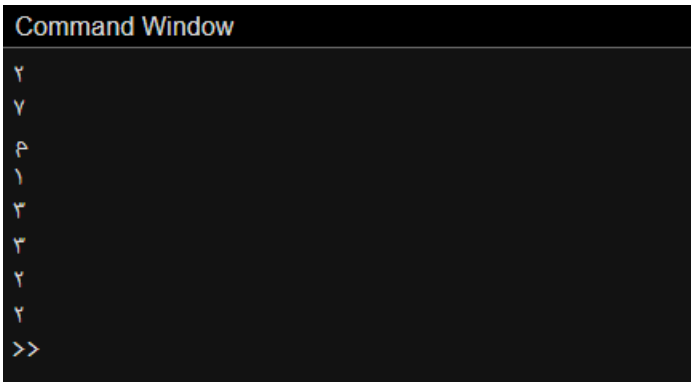
Binary

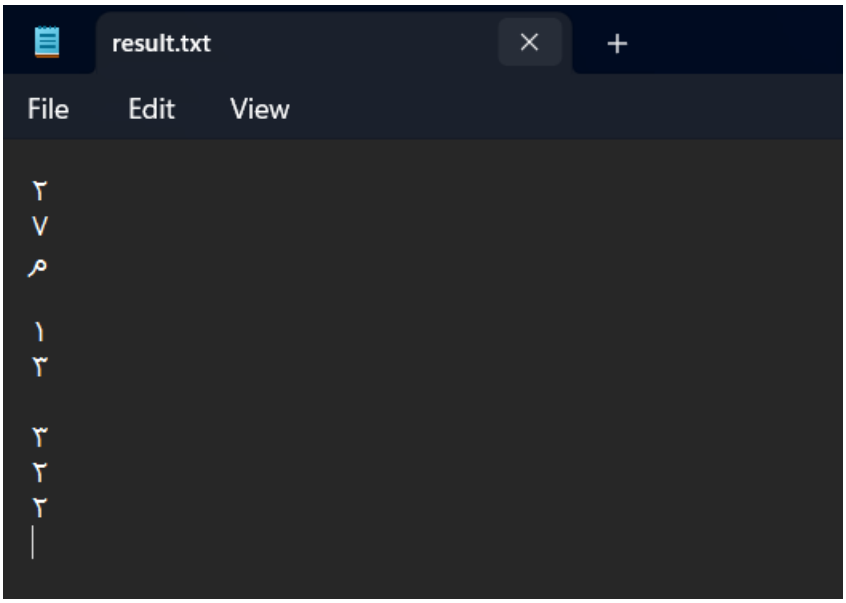


Cleaned









Part 4 : Vehicle Speed Estimation and Plate Recognition

1. Introduction

In Part 4, video frames are analyzed to measure the speed of a moving vehicle and simultaneously detect and recognize its license plate. Two selected frames from the video

are processed to estimate distance traveled, time elapsed, and thus speed. The plate in the second frame is then localized and recognized similarly to Part 3.

2. Data and Tools

- Input: video.mov.
- MATLAB built-in functions for video processing and image analysis.
- Custom function: mybinaryfun.
- Output: p4_result.txt.
- Key built-in functions: VideoReader, rgb2gray, bwareaopen, bwlabel, regionprops.

3. Algorithm / Implementation

1. Frame selection: Two frames (select1 = 6, select2 = 17) are read using VideoReader.
2. Binary mask generation:
   - Convert to grayscale.
   - Apply mybinaryfun(image,200) and invert (~image).
   - Remove small regions using bwareaopen(image,10000).
   - Label components and display bounding boxes.
3. Bounding box analysis: regionprops computes bounding boxes for moving objects.
4. Scale and motion estimation:
   - scale = stats2(2).BoundingBox(4) / stats1(2).BoundingBox(4) - 1.
   - measure_m_px = 3 / stats2(1).BoundingBox(4).
   - distance_px estimated from vertical displacement and scaled bounding-box height.
   - distance_m = distance_px × measure_m_px.
   - time_s = (select2 - select1 + 1) × (1 / FrameRate).
   - speed_m_s = distance_m / time_s.
5. Plate detection in second frame:
   - The second frame is resized and processed with grayscale → mybinaryfun → bwareaopen (4000) → regionprops.
   - The same geometric rule as Part 3 is applied to extract the plate region.
6. Character segmentation and recognition:
   - Convert to grayscale, binarize, and clean with bwareaopen(500) - bwareaopen(8000).
   - Label with bwlabel, extract each region, and correlate with Persian templates.
7. Output:
   - Display recognized characters.
   - Print speed in the console: “Speed = XX m/s”.
   - Save recognized text to p4_result.txt.

4. Key Variables and Parameters

- select1 = 6, select2 = 17 → selected frames for analysis.
- FrameRate → read from video metadata.
- Threshold for mybinaryfun = 200.
- Area filtering: 10000 (vehicle detection), 4000 (plate region), 500–8000 (character isolation).
- corr2 ≥ 0.5 for template matching.

5. Results

The algorithm successfully estimated the vehicle's speed and extracted the plate from the video sequence. Sample outputs:
- Speed displayed in console: e.g., "Speed = 17 m/s".
- Recognized plate printed and saved in p4_result.txt.
Figures show bounding boxes for vehicles and plates in the processed frames.

6. Discussion

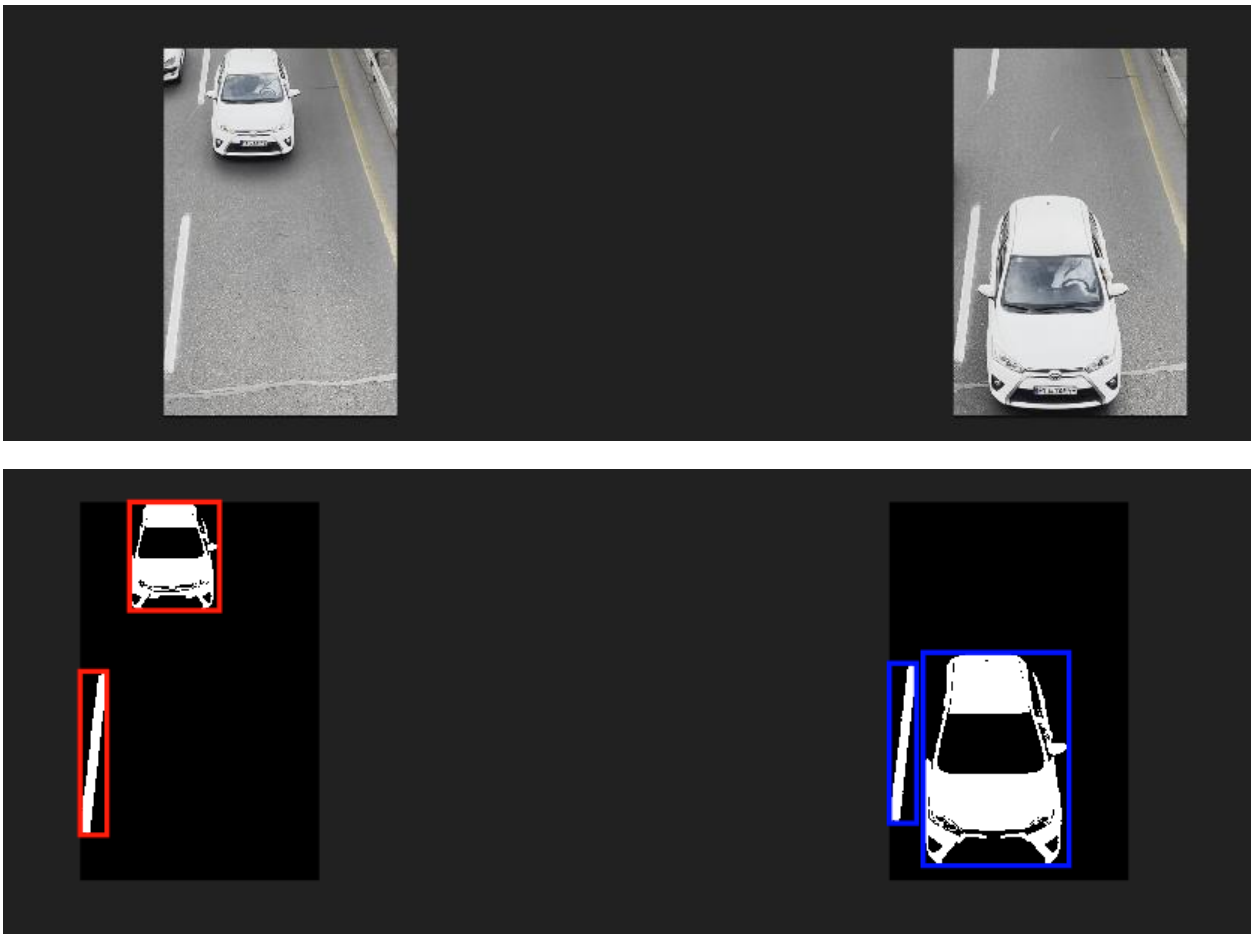
This part integrates motion analysis and image processing. The method provides a basic vehicle speed estimation based on frame displacement and bounding-box scaling. While effective for simple scenes, it assumes a fixed camera and consistent scale reference (3 meters).

Limitations:
- Sensitive to frame selection and camera calibration.
- Works best for near-frontal vehicle motion.
- Plate recognition depends on clarity and lighting of the second frame.

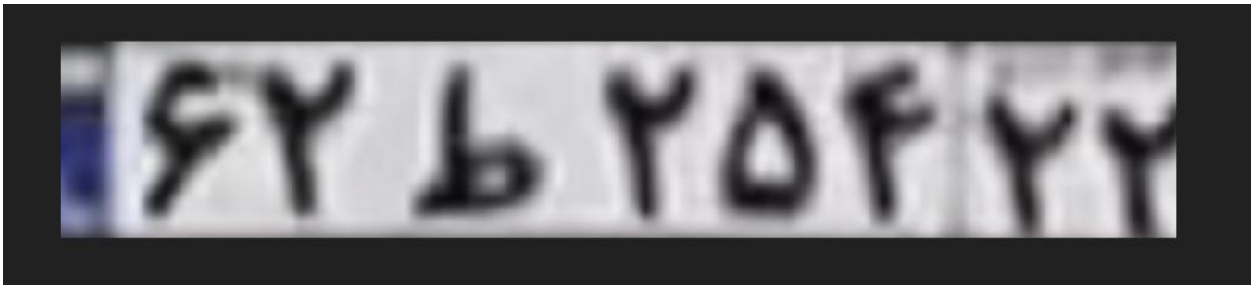
Improvements:
- Automate frame selection based on motion detection.
- Apply optical flow for continuous tracking.
- Calibrate camera to obtain real-world scale factors accurately.

7. Conclusion

Part 4 completes the system by combining motion estimation and license-plate recognition. The implementation demonstrates how signal and system concepts, combined with MATLAB's image-processing tools, can be used to measure real-world physical quantities (speed) and perform visual data analysis simultaneously.

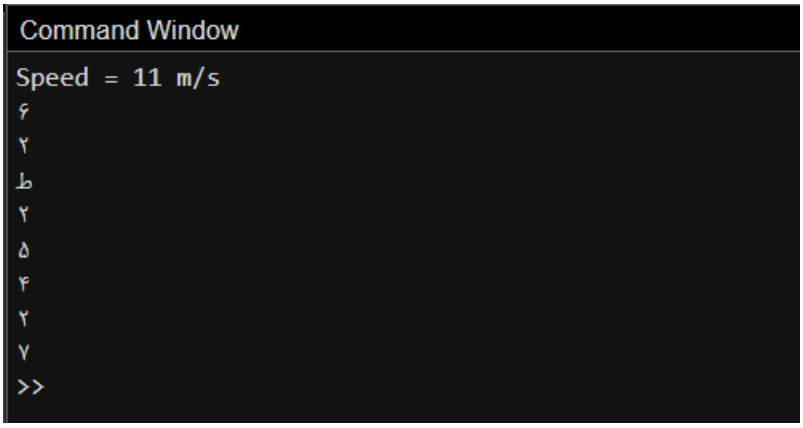





۶۲ ط ۲۵۴ ۲۲


Base
Gray
Binary
Cleaned




Command Window
Speed = 11 m/s
۶
۲
ط
۲
۵
۴
۲
۷
>>